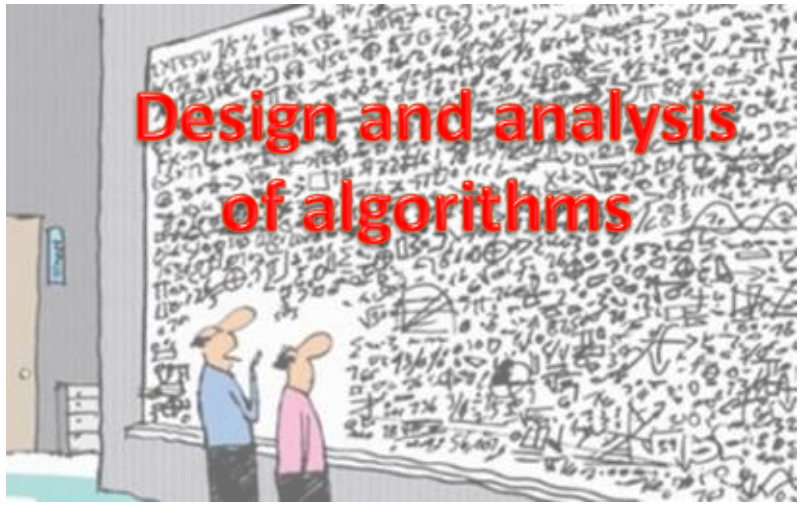


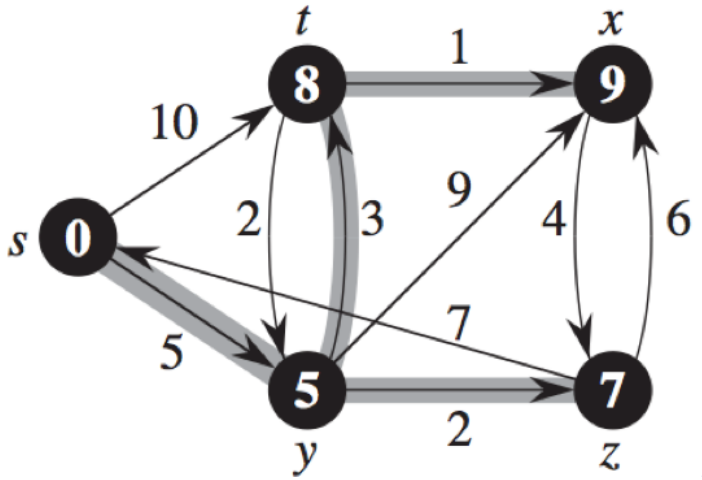
- Shortest paths II
  - DAGS
  - Dijkstra's algorithm

Objective

- Apply relaxations in a "smart" way to avoid $O(VE)$ of Bellman-Ford.
- Shortest paths in DAGs: we exploit the structure of the DAGs to do relaxations only once -> $O(V+E)$.
- Dijkstra's algorithm: smart way to relax edges only once, for general graphs with non-negative edges $\rightarrow$ $O(V \log V + E)$.
- DPV Chapter 4.
- CLRS 24.2-24.3.

What we already know

- Only relaxations **along the shortest-path tree** -> shortest distance.
- But cannot guess the shortest-path tree… unless we solve the whole problem first!
- Hmmm…

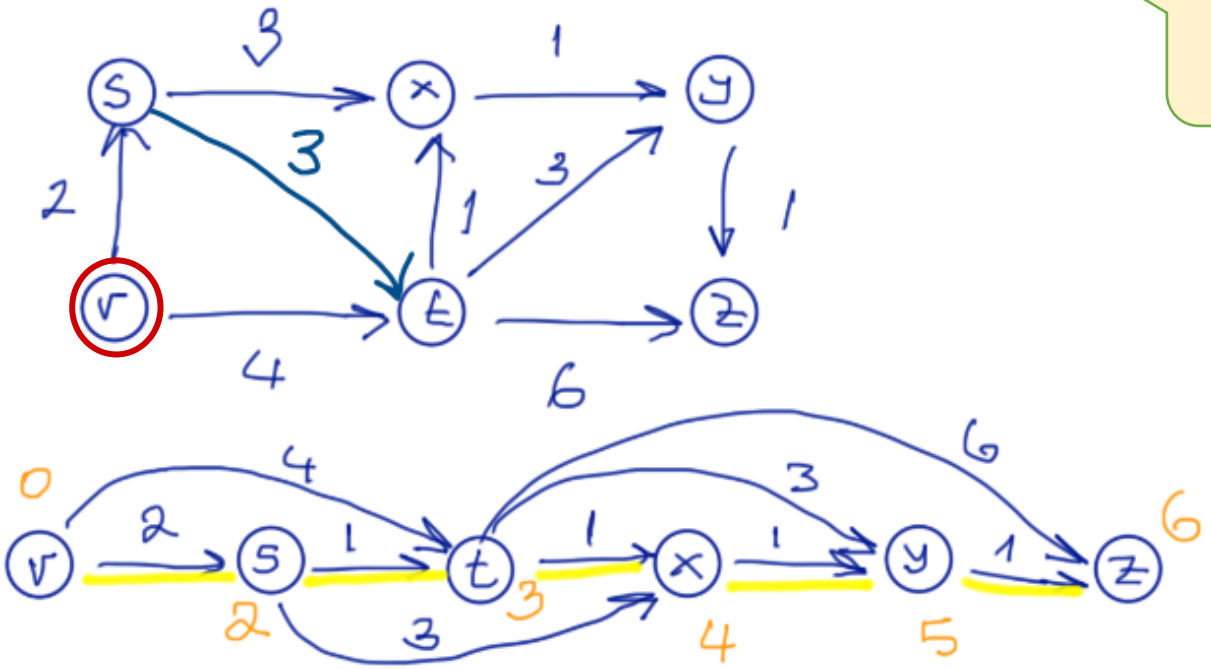


how to avoid $V \times E$ relaxations?

- Some special case where we can guess the sequence of the relaxations and relax each edge only once?

The case of directed acyclic graphs (DAGs)

- We can find the order of relaxations to be performed once by running topological sort!

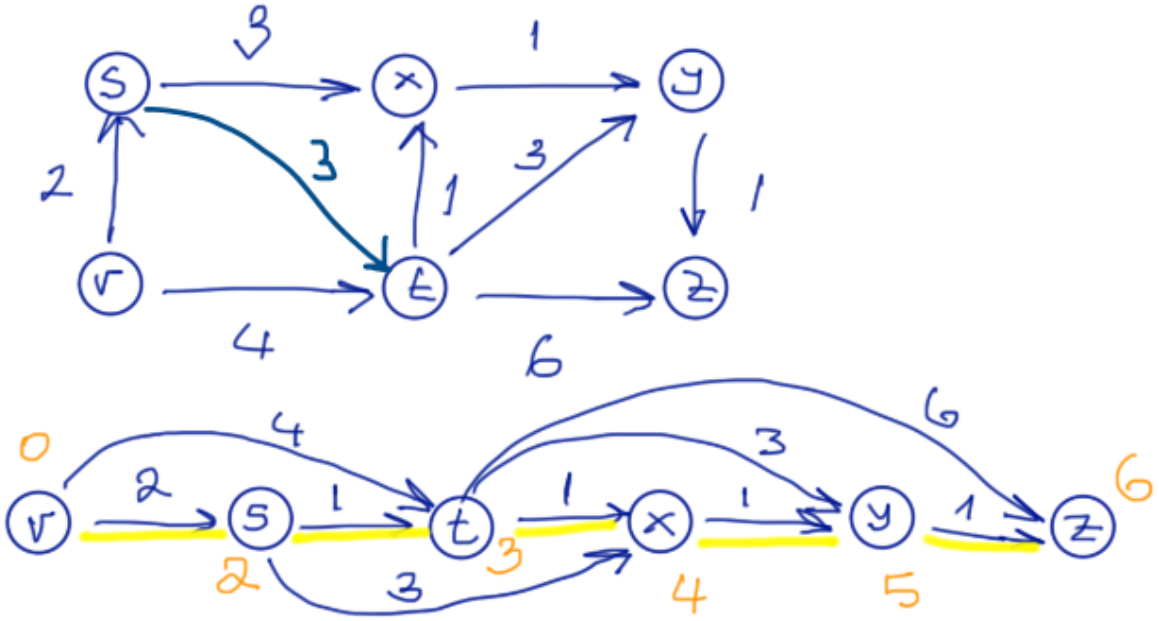


Consider the nodes in decreasing post values; relax all outgoing edges

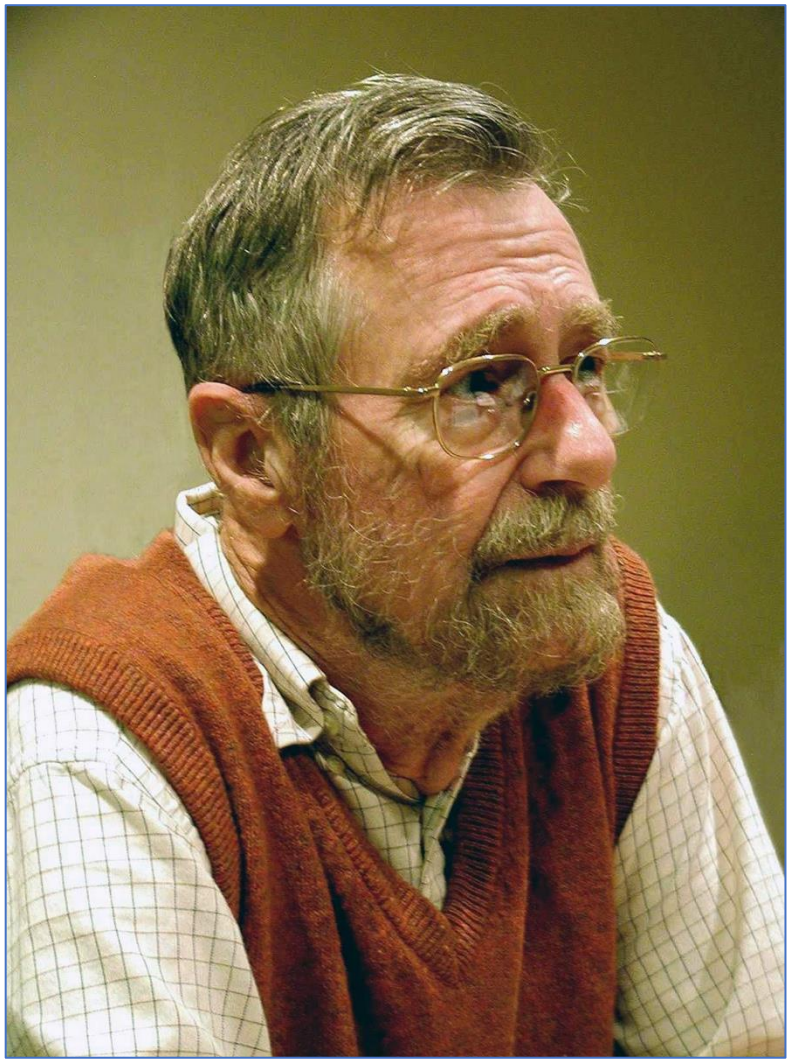
- There is no cyclic dependence in solving the Bellman equations.
- The shortest distance of a node depends only on values of previous nodes.

Complexity

- Topological sort takes time $\Theta(V + E)$ using a Depth First Search..
- Then, time spent on DAG: one visit per state, each incoming edge relaxed exactly once, hence $\Theta(V + E)$.
- Total: $\Theta(V + E)$.



Edsger W. Dijkstra (1930-2002)

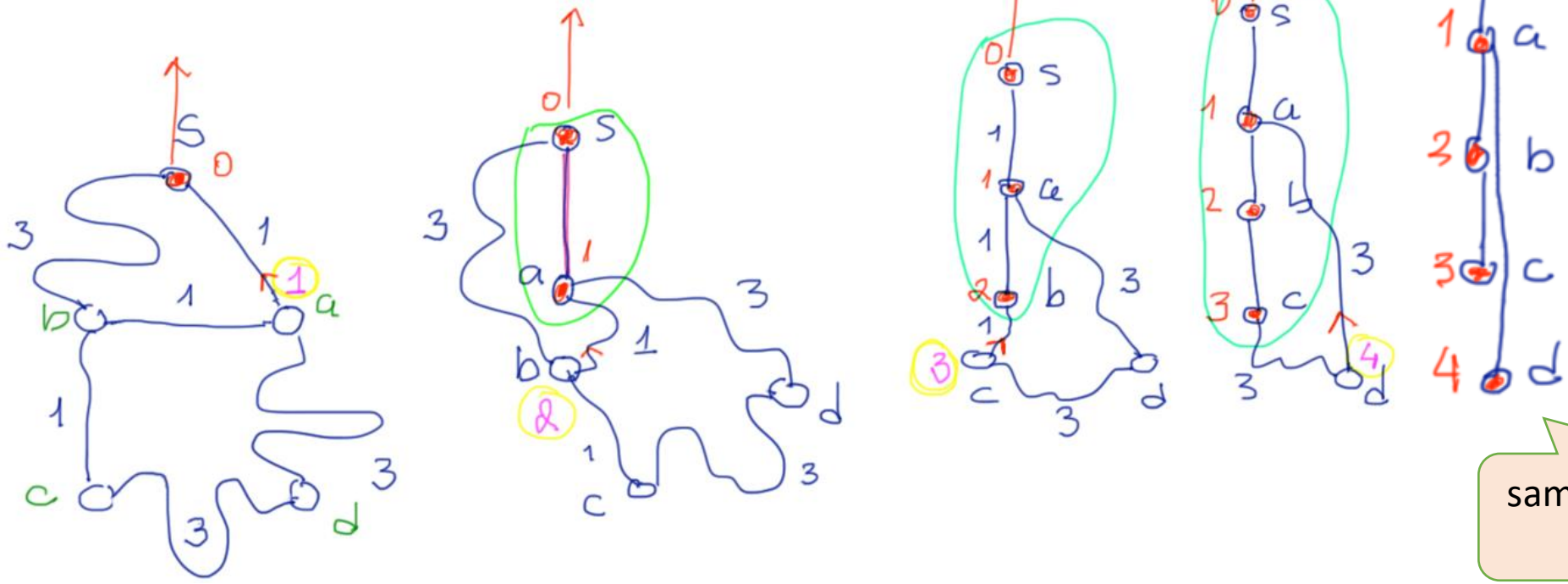


Computer Scientist with many foundational contributions to CS:

- recursion
- distributed programming
- operating systems
- structured programming
- algorithms
- programming language semantics
- correctness proofs
- etc. etc.

Dijkstra's algorithm

I start lifting node *s*:



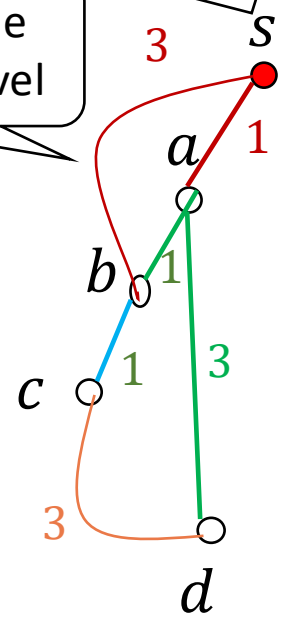
Observe: nodes enter the set S (of nodes lifted) in the order of their shortest distance to *s*, and this is through the nodes already in S.

This property is exploited in Dijkstra's algorithm!

Discovery of the shortest path tree

A signal travels at constant speed starting at time 0 from s on each outgoing link; when a node receives the signal for the first time it repeats it on its outgoing links.

length of link = time for the signal to travel



The fist time a node receives the signal is its shortest distance from s.

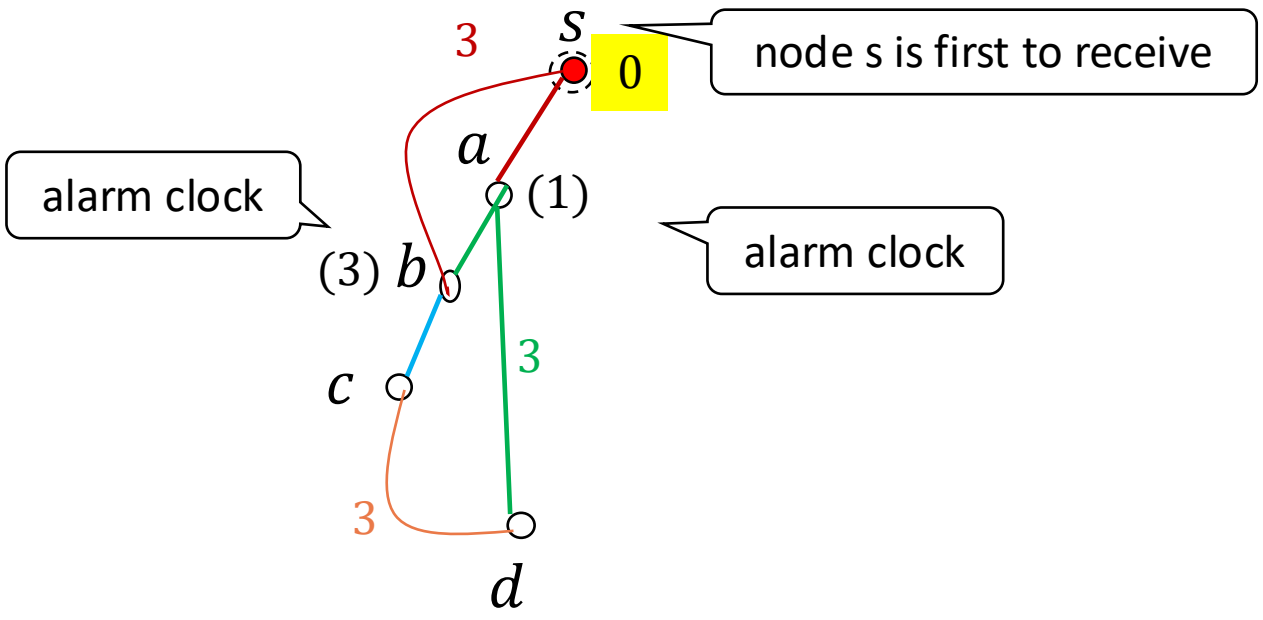
we **simulate** the system

How to construct the correct time sequence? use alarm clocks at each node

- When node i is the next node to receive the signal, it sets an alarm $t_i + w_{ij}$ to each of its neighbors j.
- A node keeps only the alarm that will go off earlier, cancels other alarms.
- At each time, the node with the earliest alarm to go off is the node that will receive the signal next.
- The link over which the signal is received is part of the SPT.

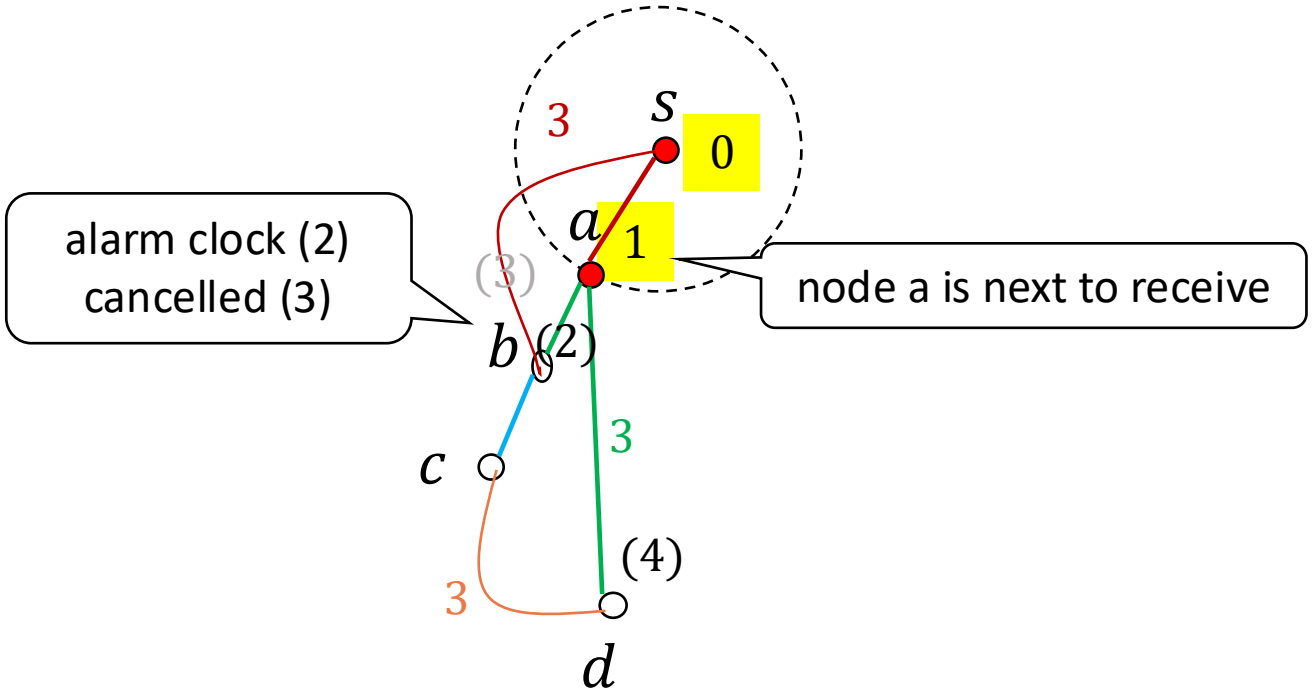
Discovery of the shortest path tree

we **simulate** the system



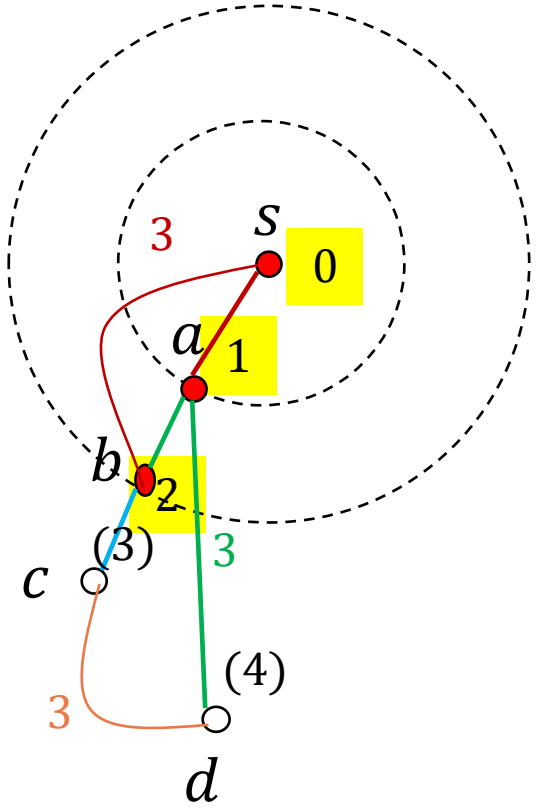
Discovery of the shortest path tree

we simulate the system

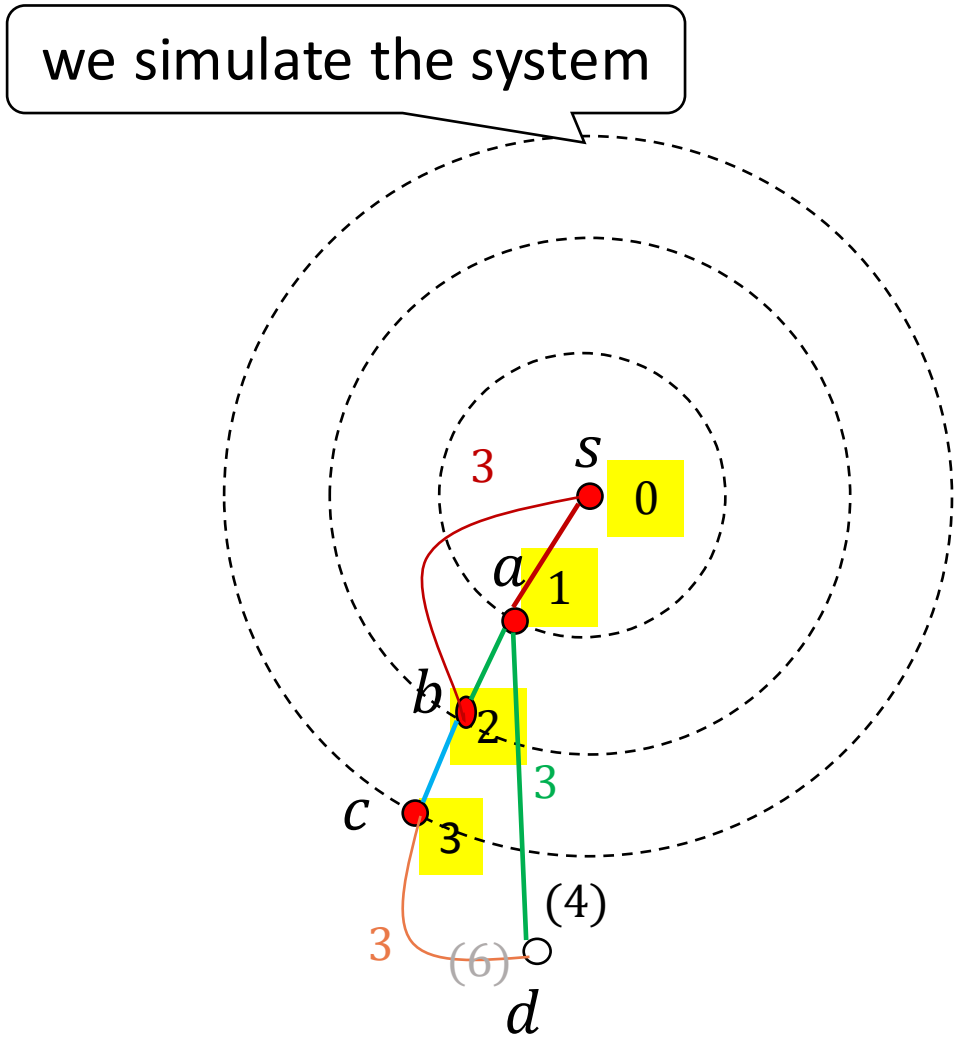


Discovery of the shortest path tree

we simulate the system

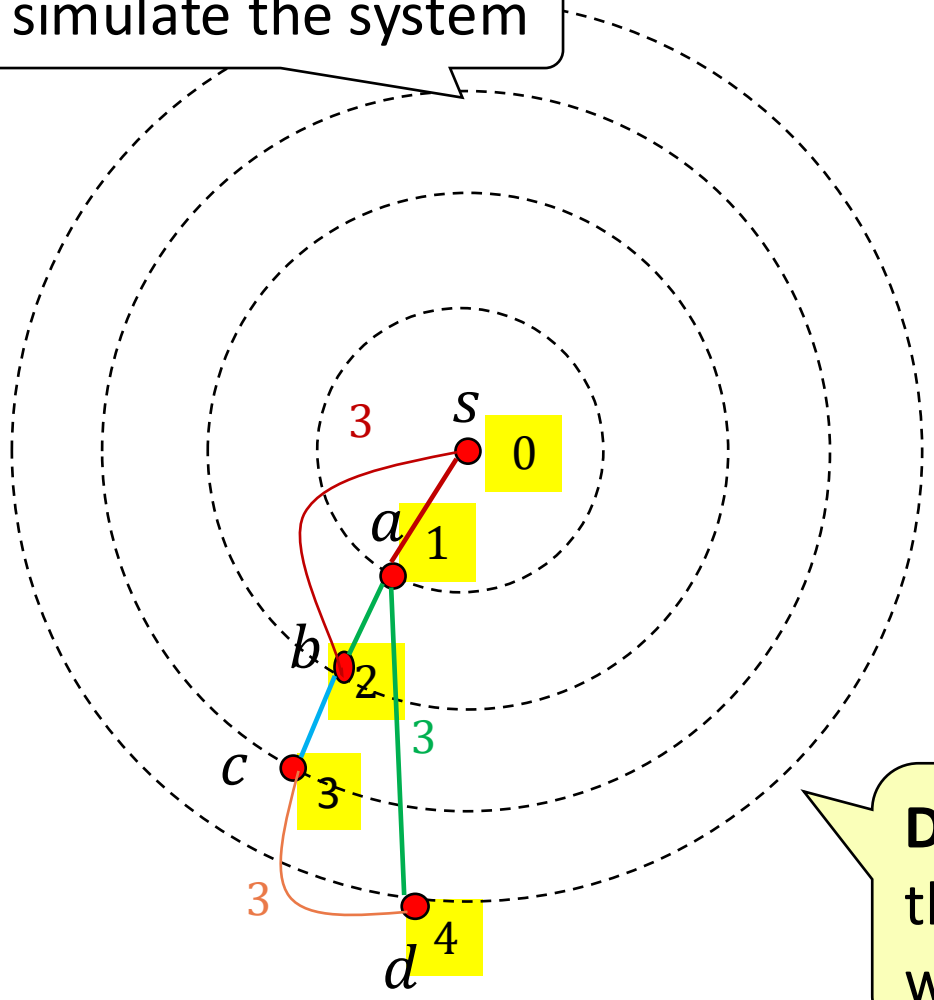


Discovery of the shortest path tree

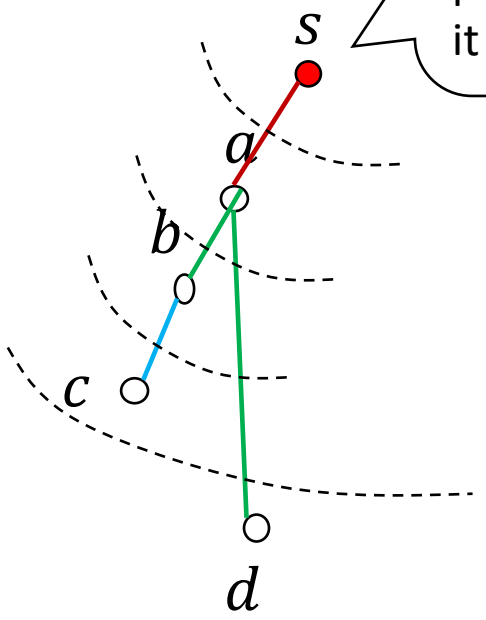


Discovery of the shortest path tree

we simulate the system

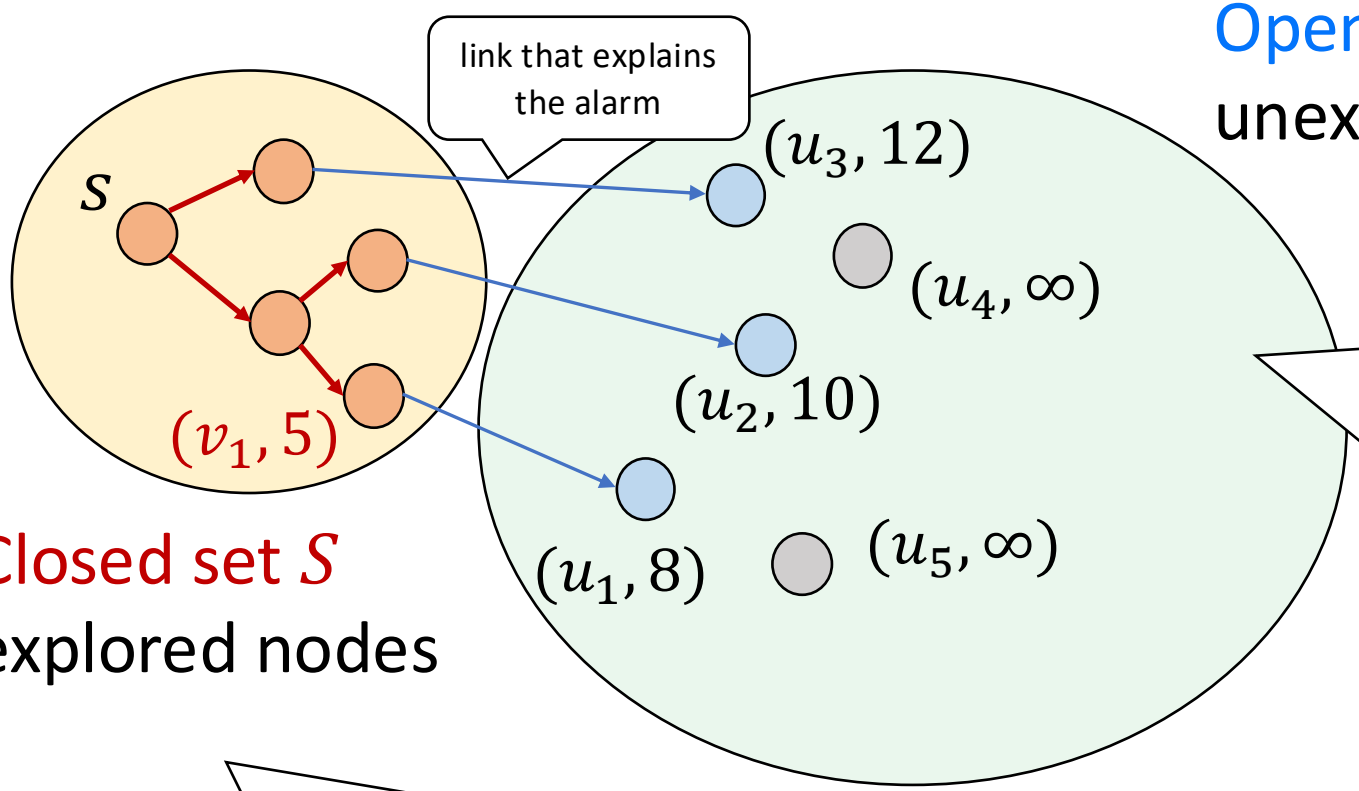


we discover the shortest path tree: a node connects to its predecessor from which it first received the signal.



Dijkstra's algorithm: simulates the system with the alarm clocks, were time = distance

Dijkstra's algorithm



Open set Q

unexplored nodes

partially explored

completely unexplored

Closed set S

explored nodes

Information in set Q:

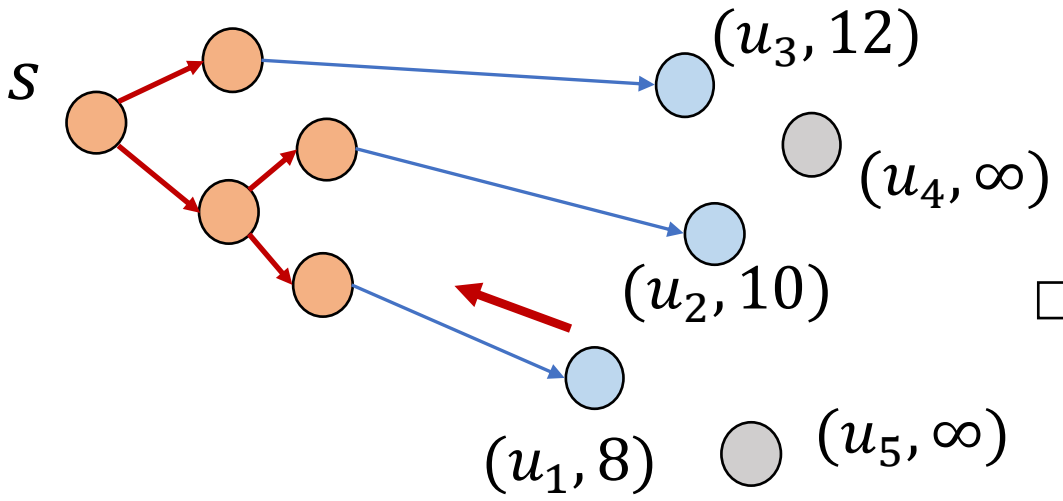
Estimates of shortest paths *through nodes* in S.

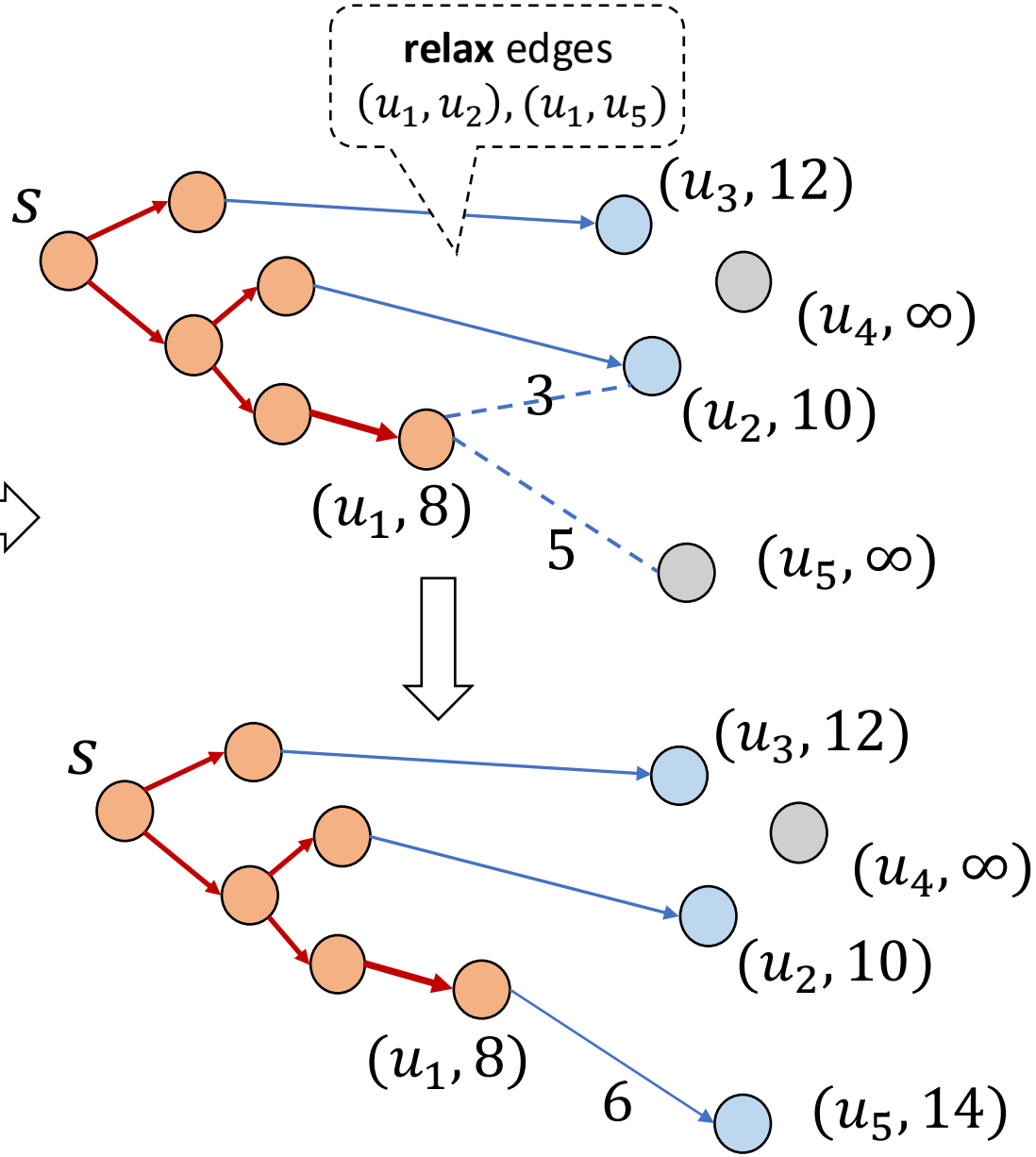
The node with the smallest estimate (smallest value of the alarm clock) has the **correct** shortest path distance.

Information in set S:

the shortest paths through nodes in S, = the **actual** shortest path tree up to a certain distance from s

Dijkstra's algorithm





relax edge (u, v):
add a new alarm in node v corresponding to (u, v), and keep the earliest alarm.

Dijkstra's algorithm (summary)

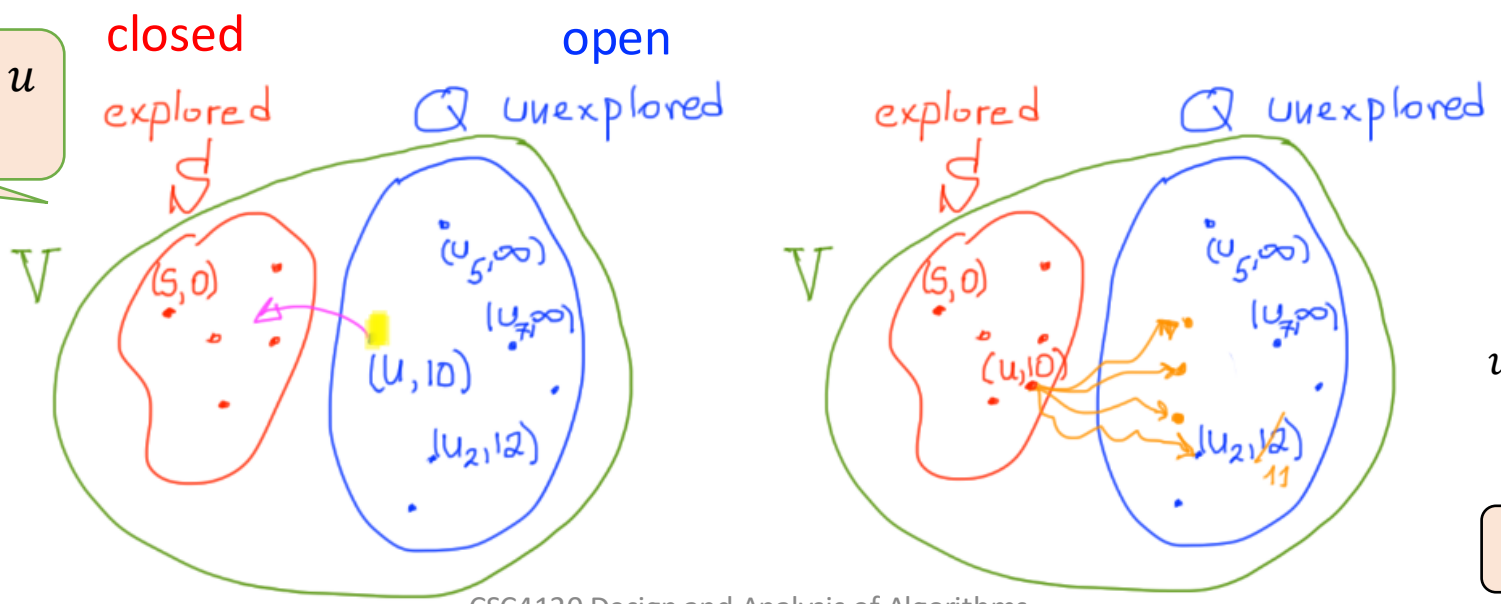
the "lifted" nodes

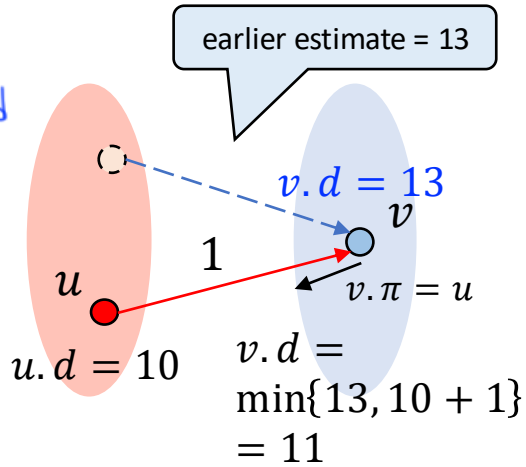
- Assumes non-negative edges $w(u, v) \geq 0 \; \forall (u, v) \in E$.
- Maintain a set S of nodes whose shortest distance have been determined:

$$v \in S \Longrightarrow v.d = \delta(v).$$

- Repeatedly select nodes **not** in S with minimum shortest path estimate through nodes already in S, add them to S, *relax* their outgoing edges.

we bring in S the node u s.t. $u.d \leq t.d, \forall t \in Q$



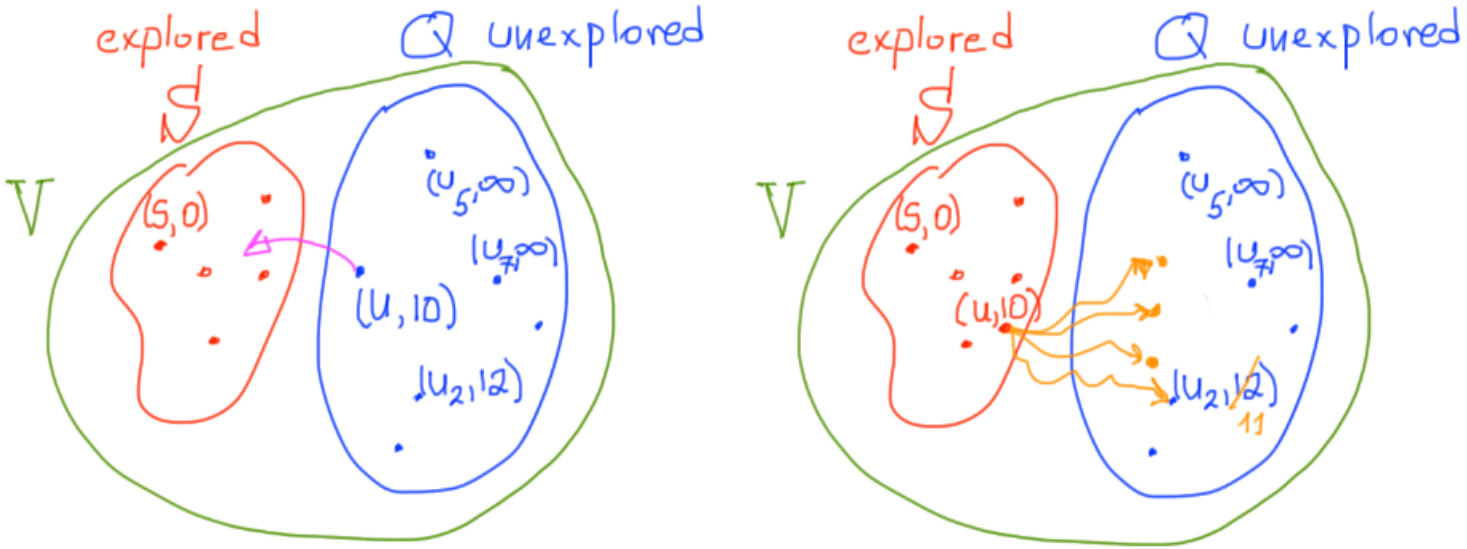


relax edge (u, v)

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V        ⇒ Q: a min-priority queue, key = u.d
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```



```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

extract_min $\times |V|$

decrease_priority value (increase actual priority) $\times |E|$

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

add_with_priority $\times |V|$

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

In a **min-priority queue implementation**

- we use as key the value of *v.d*
- operations:
  - add_with_priority() (insert_key)
  - decrease_priority() (decrease_key)
  - extract_min()

cost per type of operation

amortized

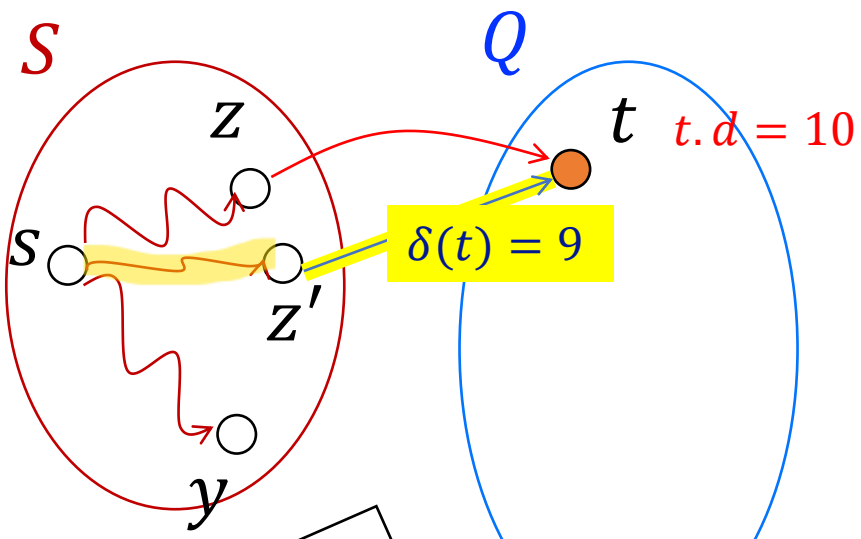
Complexity: it performs	Array	Binary-heap	Fib-heap
$\Theta(V)$ add_with_priority()	x Θ(1)	x Θ(logV)	x Θ(1)
$\Theta(V)$ extract_min()	x Θ(V)	x Θ(logV)	x Θ(logV)
$\Theta(E)$ decrease_priority()	x Θ(1)	x Θ(logV)	x Θ(1)
operations	= $\Theta(V^2+E)$ = $\Theta(V^2)$	= Θ((V+E)logV)	= Θ(VlogV+E)

Fibonacci heaps

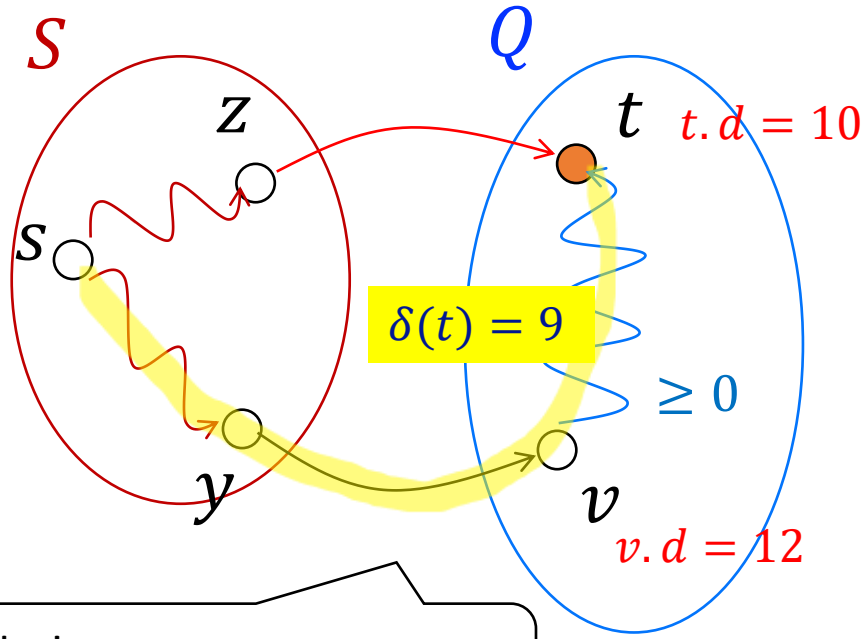
- This is a very interesting but advanced topic for students that have special interest in theoretical CS.
- https://www.cs.princeton.edu/~wayne/teaching/fibonacci-heap.pdf

Proof of correctness

- Invariant: The set S contains nodes with determined shortest path values.
- Suppose we add a new node t to S, s.t., $t.d \leq x.d, \forall x \in Q$. Then, $t.d = \delta(t)$.
- Suppose $t.d = 10$, but $\delta(t) = 9 < 10$ through the yellow path. *Contradiction!*

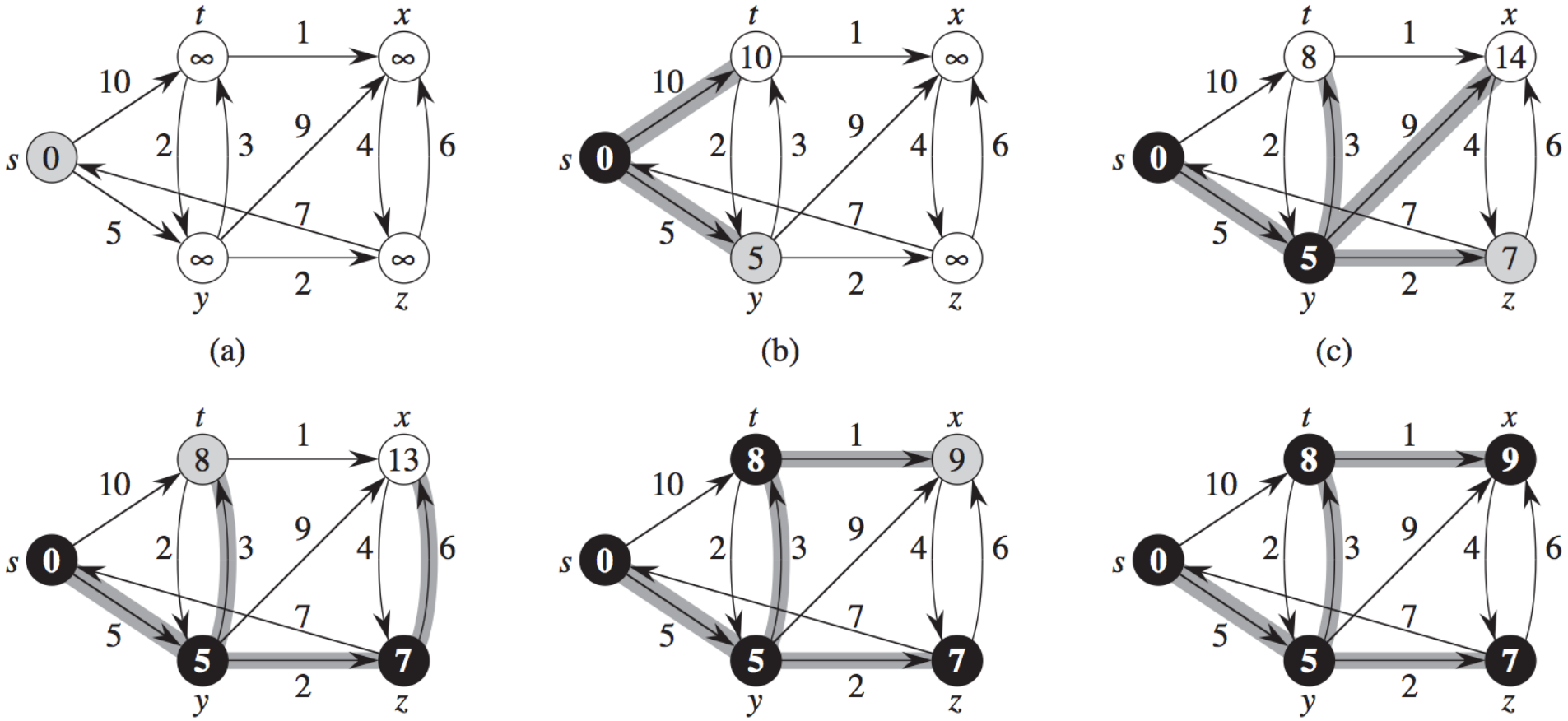


can't be!
z' has relaxed (z', t) earlier:
$t.d$ should be = 9 already.



can't be!
the length of the path is ≥ 12

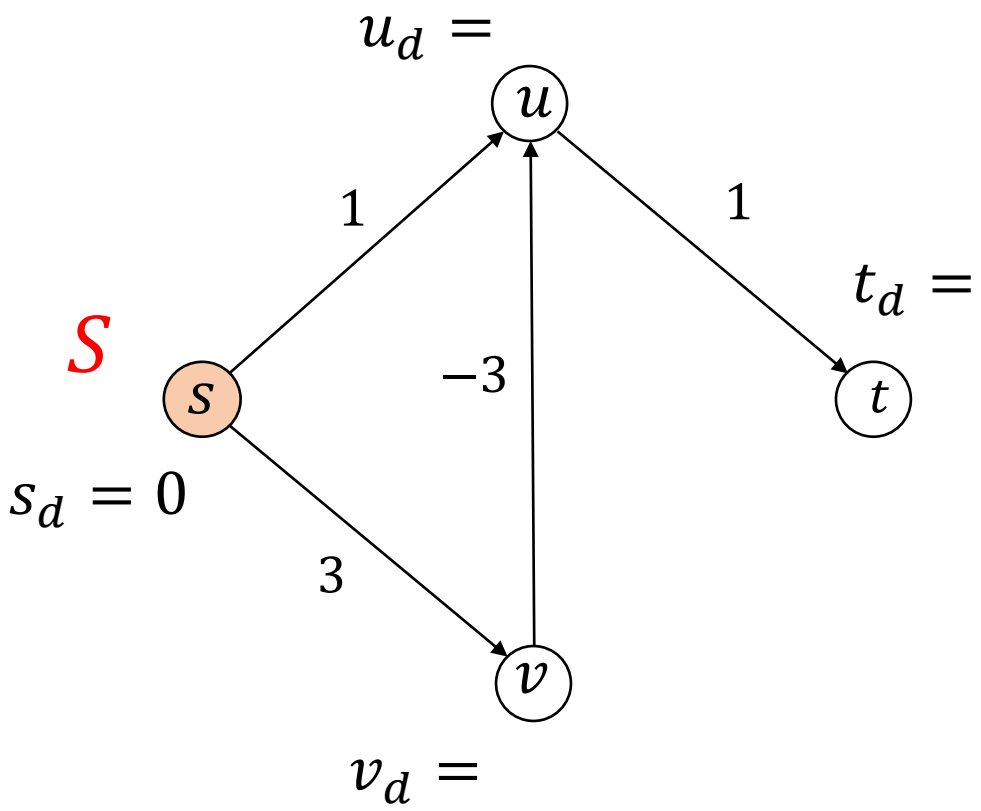
Dijkstra's algorithm example



Dijkstra with negative weights

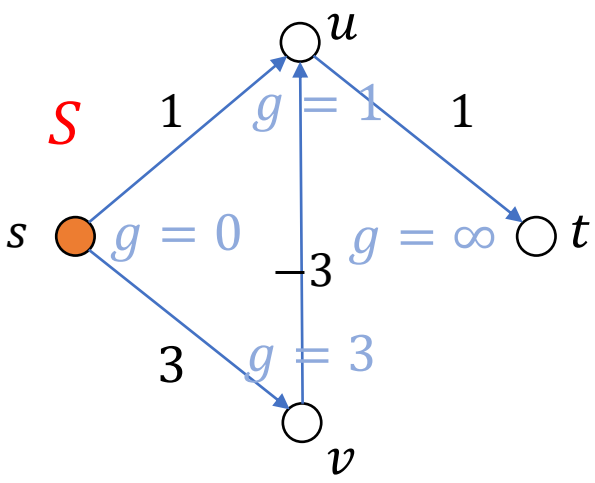
- Running Dijkstra: finding t does not imply a shortest path.
- Why? because nodes in S are not necessarily stable:
- Using a negative edge might offer better paths to nodes in S;
  - If a node in S strictly improves its estimate, it must go back to Q to be processed again and update the values of other nodes it can reach.
- Must wait until **all nodes** in S.
  - This will happen eventually (if no negative cycles), and it will give us the right shortest paths.
- Hence, no guarantee to terminate in $N - 1$ steps.

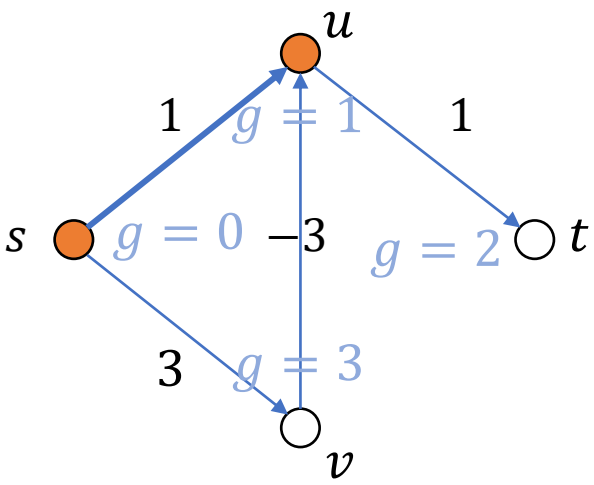
Dijkstra with negative weights



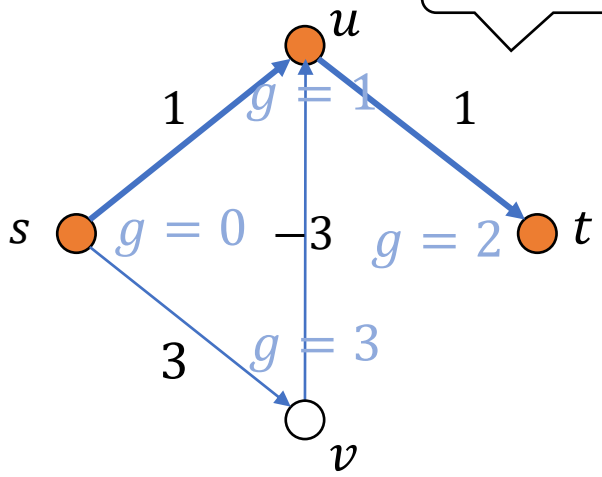
Dijkstra with negative weights

$g = x.d$

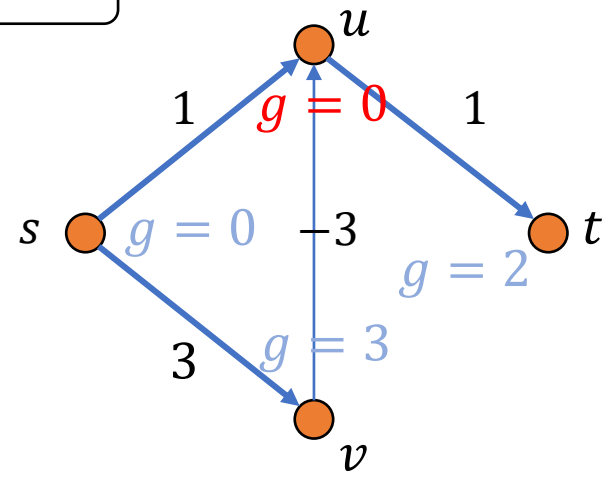




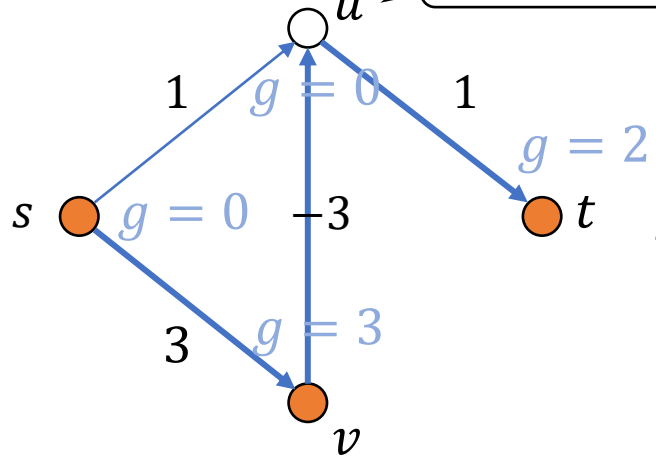
stop here?

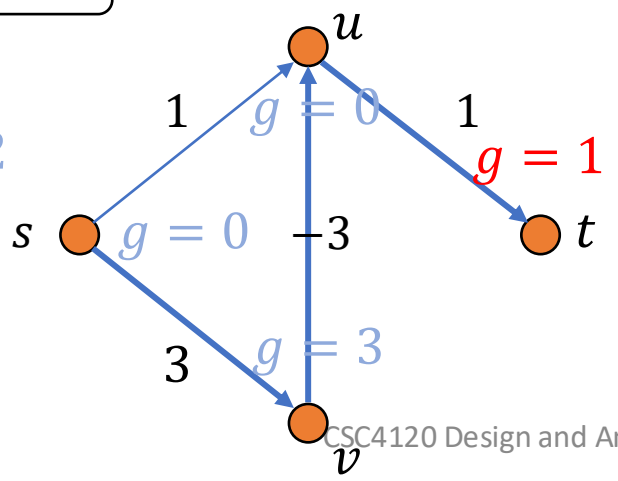


Let's continue the alg until all nodes are in S

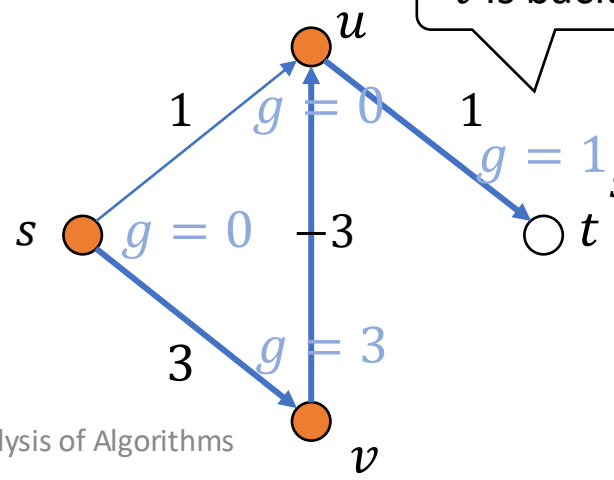


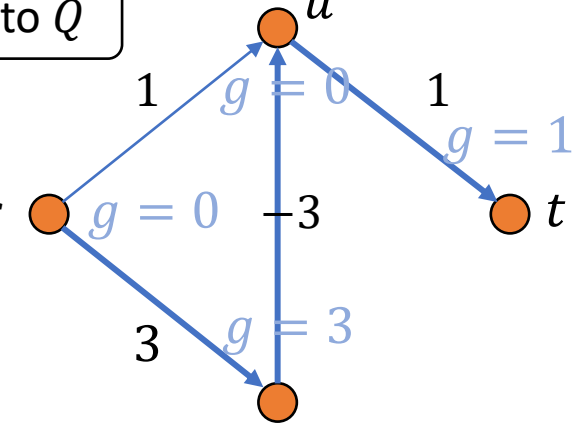
u is back to Q





t is back to Q





Conclusions

- Dijkstra's algorithm: smart way of relaxing only edges of nodes in the shortest path tree in the right order.
- Needs non-negative weights.
- Extends BFS: keep exploring nodes that are closest to s.
- Not efficient if I want to find SP to a specific node t.

